

INTRODUCTION

- Heart disease is one of the leading causes of death worldwide.
- Early prediction can help doctors provide faster treatment.
- Machine Learning helps analyze patient data efficiently.
- ML models can detect hidden medical patterns from clinical data.
- This project predicts whether a patient has heart disease or not.

Technologies Used

Python, Machine Learning, XGBoost , Random Forest
Logistic Regression, KNN

OBJECTIVES

- Build a binary classification model
- Predict heart disease using clinical features
- Compare multiple ML algorithms
- Improve prediction accuracy
- Handle missing values and mixed data
- Save best model for future deployment

DATASET INFORMATION

Dataset Used

UCI Heart Disease Dataset

- Source: Kaggle / UCI Repository
- Total Records: 920
- Total Features: 15
- Data collected from:
 - Cleveland
 - Hungary
 - Switzerland
 - VA Long Beach

Important Features

- Age
- Blood Pressure
- Cholesterol
- Chest Pain Type
- ECG Results
- Heart Rate

Feature	Type	Description
age	Numerical	Patient age in years
sex	Categorical	Male / Female
cp	Categorical	Chest pain type: typical angina, atypical angina, non-anginal, asymptomatic
trestbps	Numerical	Resting blood pressure (mm Hg)
chol	Numerical	Serum cholesterol (mg/dl)
fbs	Categorical	Fasting blood sugar > 120 mg/dl (True/False)
restecg	Categorical	Resting ECG results
thalch	Numerical	Maximum heart rate achieved
exang	Categorical	Exercise-induced angina (True/False)
oldpeak	Numerical	ST depression induced by exercise relative to rest
slope	Categorical	Slope of peak exercise ST segment
ca	Numerical	Number of major vessels colored by fluoroscopy (0-3)
thal	Categorical	Thalassemia: normal, fixed defect, reversable defect
target	Binary	0 = No disease, 1 = Disease present

DATA CLEANING

```
import pandas as pd
import numpy as np

# 1. Load
df = pd.read_csv('/kaggle/input/datasets/shantanusharma23410/heart-disease-csv/heart_disease_uci.csv')

# 2. Basic Info
print("Shape:", df.shape)
print("\nColumn Names:\n", df.columns.tolist())
print("\nData Types:\n", df.dtypes)
print("\nFirst 5 Rows:\n", df.head())

# 3. Fix Target Column
df['num'] = df['num'].apply(lambda x: 1 if x > 0 else 0)
df.rename(columns={'num': 'target'}, inplace=True)
print("\nTarget Distribution:\n", df['target'].value_counts())

# 4. Check Missing Values
print("\nMissing Values:\n", df.isnull().sum())
print("\nMissing %:\n", (df.isnull().sum() / len(df) * 100).round(2))

# 5. Drop Useless Columns
# 'id' is just a row number – no predictive value
df.drop(columns=['id'], inplace=True, errors='ignore')
```

Shape: (920, 16)

Column Names:

['id', 'age', 'sex', 'dataset', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalch', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'num']

First 5 Rows:

	id	age	sex	dataset	cp	trestbps	chol	fbs	restecg	thalch	exang	oldpeak	slope	ca	thal	num
0	1	63	Male	Cleveland	typical angina	145.0	233.0	True	lv hypertrophy	150.0	False	2.3	downsloping	0.0	fixed defect	0
1	2	67	Male	Cleveland	asymptomatic	160.0	286.0	False	lv hypertrophy	108.0	True	1.5	flat	3.0	normal	2
2	3	67	Male	Cleveland	asymptomatic	120.0	229.0	False	lv hypertrophy	129.0	True	2.6	flat	2.0	reversable defect	1
3	4	37	Male	Cleveland	non-anginal	130.0	250.0	False	normal	187.0	False	3.5	downsloping	0.0	normal	0
4	5	41	Female	Cleveland	atypical angina	130.0	204.0	False	lv hypertrophy	172.0	False	1.4	upsloping	0.0	normal	0

```

# 6. Fix Missing Values
# Numerical columns == fill with median
num_cols = df.select_dtypes(include=['float64', 'int64']).columns.tolist()
num_cols = [c for c in num_cols if c != 'target'] # exclude target
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# Categorical columns == fill with mode
cat_cols = df.select_dtypes(include=['object']).columns.tolist()
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

# 7. Verify No Missing Values Left
print("\nMissing After Cleaning:\n", df.isnull().sum())

# 8. Final Shape
print("\nFinal Shape:", df.shape)
print("\nCleaned Data Sample:\n", df.head()).

```

	age	sex	dataset	cp	trestbps	chol	fbs	restecg	thalch	exang	oldpeak	slope	ca	thal	target
0	63	Male	Cleveland	typical angina	145.0	233.0	True	lv hypertrophy	150.0	False	2.3	downsloping	0.0	fixed defect	0
1	67	Male	Cleveland	asymptomatic	160.0	286.0	False	lv hypertrophy	108.0	True	1.5	flat	3.0	normal	1
2	67	Male	Cleveland	asymptomatic	120.0	229.0	False	lv hypertrophy	129.0	True	2.6	flat	2.0	reversable defect	1
3	37	Male	Cleveland	non-anginal	130.0	250.0	False	normal	187.0	False	3.5	downsloping	0.0	normal	0
4	41	Female	Cleveland	atypical angina	130.0	204.0	False	lv hypertrophy	172.0	False	1.4	upsloping	0.0	normal	0

FEATURES AND TARGET

```
# Split Features & Target
X = df.drop('target', axis=1)
y = df['target']

print("X shape:", X.shape)
print("y shape:", y.shape)
print("\ny distribution:\n", y.value_counts())
```

X shape: (920, 14)

y shape: (920,)

y distribution:

target

1 509

0 411

Name: count, dtype: int64

TRAIN-TEST SPLIT

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

X_train shape: (736, 14)

X_test shape: (184, 14)

y_train shape: (736,)

y_test shape: (184,)

ONE HOT ENCODING

```
# One Hot Encoding  
df = pd.get_dummies(df, drop_first=False)  
  
print(df.shape)  
print(df.head())
```

(920, 28)

STANDARD SCALING

```
from sklearn.preprocessing import StandardScaler  
  
# Standard Scaling  
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

TRAIN LOGISTIC REGRESSION

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Train Logistic Regression
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)

# Predictions
y_pred = lr.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Accuracy: 0.8206521739130435

Classification Report:

	precision	recall	f1-score	support
0	0.76	0.81	0.79	75
1	0.87	0.83	0.85	109
accuracy			0.82	184
macro avg	0.81	0.82	0.82	184
weighted avg	0.82	0.82	0.82	184

Confusion Matrix:

[[61 14]

[19 90]]

TRAIN RANDOM FOREST

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Train Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Predictions
y_pred_rf = rf.predict(X_test)

# Evaluation
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))

# Compare both models
print("\n--- Model Comparison ---")
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred))
print("Random Forest Accuracy:      ", accuracy_score(y_test, y_pred_rf))
```

Random Forest Accuracy: 0.8369565217391305

--- Model Comparison ---

Logistic Regression Accuracy: 0.8206521739130435

Random Forest Accuracy: 0.8369565217391305

TRAIN KNN

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Train KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# Predictions
y_pred_knn = knn.predict(X_test)

# Evaluation
print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))

# Compare all 3 models
print("\n--- Model Comparison ---")
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred))
print("Random Forest Accuracy:      ", accuracy_score(y_test, y_pred_rf))
print("KNN Accuracy:                  ", accuracy_score(y_test, y_pred_knn)).
```

KNN Accuracy: 0.8369565217391305

--- Model Comparison ---

Logistic Regression Accuracy: 0.8206521739130435

Random Forest Accuracy: 0.8369565217391305

KNN Accuracy: 0.8369565217391305

TRAIN XGBOOST

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Train XGBoost
xgb = XGBClassifier(n_estimators=100, random_state=42, eval_metric='logloss')
xgb.fit(X_train, y_train)

# Predictions
y_pred_xgb = xgb.predict(X_test)

# Evaluation
print("XGBoost Accuracy:", accuracy_score(y_test, y_pred_xgb))
print("\nClassification Report:\n", classification_report(y_test, y_pred_xgb))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_xgb))

# Compare all 4 models
print("\n--- Model Comparison ---")
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred))
print("Random Forest Accuracy:      ", accuracy_score(y_test, y_pred_rf))
print("KNN Accuracy:                  ", accuracy_score(y_test, y_pred_knn))
print("XGBoost Accuracy:              ", accuracy_score(y_test, y_pred_xgb))
```

XGBoost Accuracy: 0.842391304347826

--- Model Comparison ---

Logistic Regression Accuracy: 0.8206521739130435

Random Forest Accuracy: 0.8369565217391305

KNN Accuracy: 0.8369565217391305

XGBoost Accuracy: 0.842391304347826

MODEL COMPARISON

```
from sklearn.metrics import accuracy_score, roc_auc_score, f1_score

# All model predictions
models = {
    'Logistic Regression': lr.predict(X_test),
    'Random Forest'      : rf.predict(X_test),
    'KNN'                 : knn.predict(X_test),
    'XGBoost'             : xgb.predict(X_test)
}

# Evaluation
print(f"{'Model':<25} {'Accuracy':>10} {'F1 Score':>10} {'AUC-ROC':>10}")
print("-" * 60)

for name, y_pred_model in models.items():
    acc = accuracy_score(y_test, y_pred_model)
    f1  = f1_score(y_test, y_pred_model)
    auc = roc_auc_score(y_test, y_pred_model)
    print(f"{'name':<25} {'acc':>10.4f} {'f1:>10.4f} {'auc:>10.4f}")
```

Model	Accuracy	F1 Score	AUC-ROC

Logistic Regression	0.8207	0.8451	0.8195
Random Forest	0.8370	0.8585	0.8374
KNN	0.8370	0.8585	0.8374
XGBoost	0.8424	0.8612	0.8462

SELECTING BEST MODEL

```
import pickle
from sklearn.metrics import accuracy_score

# Find best model
models = {
    'Logistic Regression': lr,
    'Random Forest'       : rf,
    'KNN'                  : knn,
    'XGBoost'              : xgb
}

# Pick best based on accuracy
best_name = max(models, key=lambda name: accuracy_score(y_test, models[name].predict(X_test)))
best_model = models[best_name]

print(f"Best Model: {best_name}")
print(f"Accuracy: {accuracy_score(y_test, best_model.predict(X_test)):.4f}")

# Save best model and scaler
pickle.dump(best_model, open("heart_model.pkl", "wb"))
pickle.dump(scaler, open("scaler.pkl", "wb"))

print("\nSaved: heart_model.pkl")
print("Saved: scaler.pkl")
```

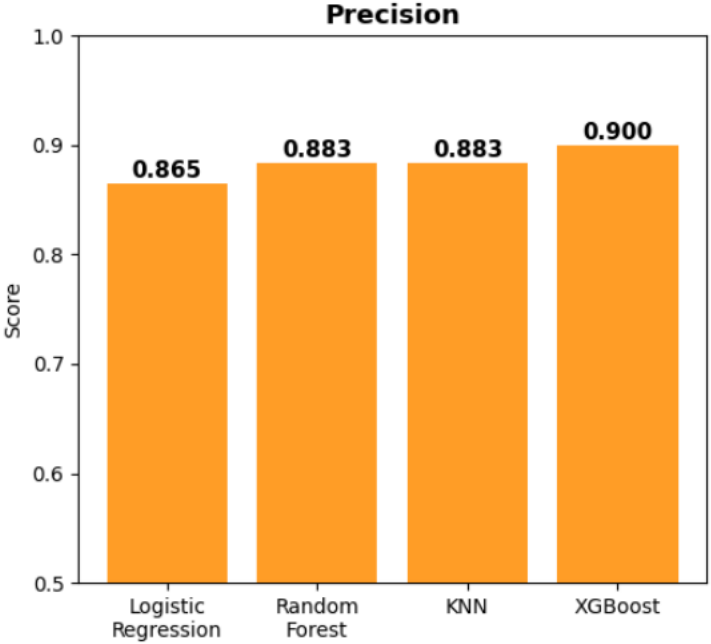
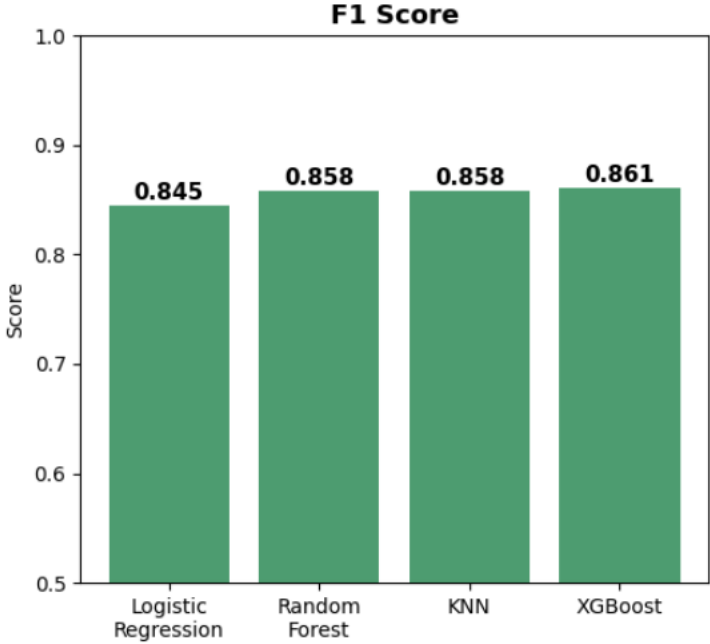
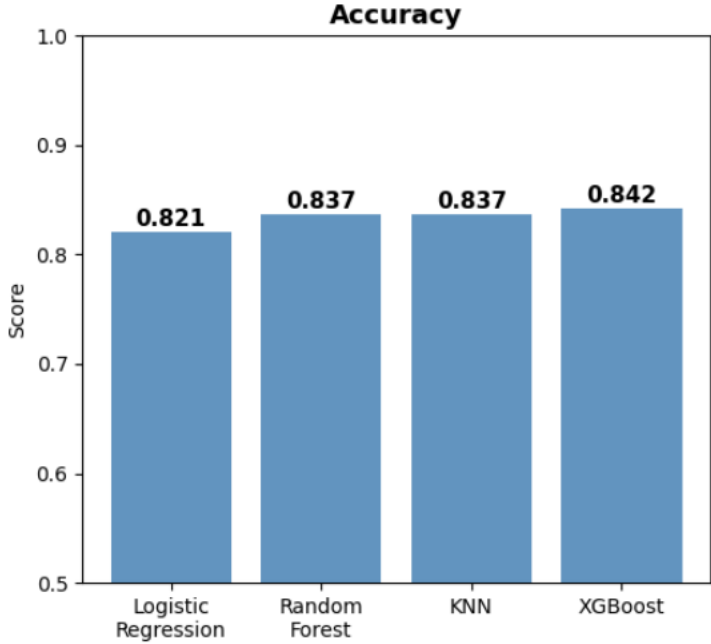
Best Model: XGBoost

Accuracy: 0.8424

Saved: heart_model.pkl

Saved: scaler.pkl

Model Comparison



SAMPLE PREDICTION

Sample #	Age	Sex	Chest Pain Type	Resting BP	Cholesterol	Max HR	Actual	Predicted	Result
1	63	Male	Typical angina	145	233	150	0	0	✓ Correct
2	67	Male	Asymptomatic	160	286	108	1	1	✓ Correct
3	67	Male	Asymptomatic	120	229	129	1	1	✓ Correct
4	37	Male	Non-anginal	130	250	187	0	0	✓ Correct
5	41	Female	Atypical angina	130	204	172	0	1	✗ Wrong
6	56	Male	Asymptomatic	130	256	142	1	1	✓ Correct
7	62	Female	Asymptomatic	140	268	160	0	0	✓ Correct
8	57	Female	Asymptomatic	120	354	163	1	0	✗ Wrong
9	44	Male	Atypical angina	120	220	188	0	0	✓ Correct
10	52	Male	Non-anginal	125	212	168	1	1	✓ Correct

Summary: 8/10 correct on this sample — consistent with the overall 84.24% test accuracy.

Prediction labels: 0 = No Heart Disease, 1 = Heart Disease Present

CHALLENGES

1. Missing Values

- Many columns contained large amounts of missing data.
- Required careful preprocessing and imputation.

2. Data Imbalance

- Number of disease and non-disease cases were not perfectly balanced.
- Could affect model prediction performance.

3. Feature Encoding

- Categorical medical data needed conversion into numerical format using One-Hot Encoding.

4. Small Dataset Size

- Dataset contained only 920 records.
- Limited data can reduce generalization capability.

5. Model Selection

- Different ML models produced different accuracies.
- Choosing the best model required comparison using multiple metrics.

6. Computational Complexity

- XGBoost and Random Forest required more training time compared to Logistic Regression.

Why XGBoost Outperforms Other Models

XGBoost's edge in heart disease prediction comes down to one core advantage: it **learns from its own mistakes**, iteratively refining predictions where other models fall short.

Boosting

Builds trees sequentially — each new tree corrects errors from the previous one

Complex Patterns

Captures nonlinear relationships in heart disease data that Logistic Regression misses

Mixed Data

Handles numerical features, encoded categoricals, and missing values natively

Regularization

Built-in L1/L2 regularization prevents overfitting to training data

XGBoost vs Other Models

Not all ensemble methods are equal. What sets XGBoost apart is its **sequential error correction** and **optimized engineering** — including parallel processing and efficient tree pruning — making it both accurate and fast.

✔ XGBoost also performs automatic **feature importance ranking**, identifying which medical indicators (e.g., cholesterol, age, chest pain type) drive predictions most — a critical advantage in clinical contexts.

Model	Key Limitation
Logistic Regression	Linear relationships only
KNN	Sensitive to feature scaling and noise
Random Forest	Trees are independent — no error learning
XGBoost ✔	Learns from previous mistakes iteratively